WASP Software Engineering Assignment

Yufei Wu
yufeiwu@cs.umu.se

June 2026

1 Introduction

My research area is software security, and I am currently working on dynamic taint analysis (DTA). DTA is a runtime technique that tracks how sensitive data propagates through a program: it associates labels with designated sources and propagates those labels along data flows until the data reaches a sink. Because DTA is increasingly entrusted with safeguarding sensitive data in security-critical contexts, from privacy auditing to runtime compliance enforcement, the integrity of its underlying taint metadata has direct consequences for the security and compliance guarantees it is meant to provide.

To track taint, DTA tools must store this metadata alongside program data, and existing systems do so through three strategies: shadowing, which places tags adjacent to program values; mirroring, which keeps tags in a separate memory region within the same process; and decoupling, which places metadata outside the process entirely. Prior work showed that shadowing can be silently corrupted through type confusion, while mirroring was assumed safer because its isolated metadata has no references from program code.

In my recent work, we demonstrate for the first time that this assumption does not hold: isolated (mirrored) metadata storage can also be defeated through type confusion, by constructing a discovery mechanism that locates heap-resident data structures with no application-level references. We propose a taxonomy of DTA metadata storage strategies and identify the process address space, rather than the way metadata is organized, as the fundamental security boundary for metadata integrity. We then discuss implications for DTA design and outline mitigation directions, including off-process metadata storage and hardware-based memory protection.

2 Lecture Principles

2.1 Verification vs. Validation

Verification asks whether we are building the product right; validation asks whether we are building the right product [2]. Verification checks an implementation against its specification, while validation checks the specification (or the implementation) against the actual needs of stakeholders. This distinction, introduced in Lecture 2, maps directly onto the evaluation gap that motivates my research.

Existing DTA tools only verify whether they meet their specification. They check whether taint propagates correctly across operations and whether instrumentation covers the intended code paths. MirrorTaint, for instance, is evaluated on taint propagation accuracy across Java bytecode operations; Galette measures overhead on benchmark suites; Fontus tests cross-component taint persistence. All three confirm that the tools do what they claim to do under normal operating conditions.

What is missing is validation. DTA tools exist to provide security guarantees about information flow: they are deployed for privacy auditing, compliance enforcement, and vulnerability detection. Yet the

designers of these tools evaluated only whether taint propagation functions correctly, not whether the security guarantees themselves hold under adversarial conditions. A tool can propagate every taint label flawlessly and still be useless if an attacker can silently erase those labels from memory.

Our work addresses this validation gap. We show that a type-confusion attacker operating within the same process can locate and destroy mirrored taint metadata that has no application-level references, using a heap navigation chain through HotSpot's internal Metaspace structures. The tools pass verification (taint propagates correctly) but fail validation (the security guarantee does not survive the conditions the tool is meant to protect against). Our taxonomy of DTA metadata storage strategies (shadowing, mirroring, and decoupling) is organized around this validation question, evaluating whether metadata integrity holds under each strategy when the process is compromised.

2.2 Confirmation Bias

Confirmation bias is the tendency to seek information that supports existing beliefs while underweighting disconfirmatory evidence [1]. In software engineering, this bias manifests as a preference for tests that confirm a system works over tests that attempt to break it.

This pattern describes the evaluation culture around DTA tools. Every major DTA tool has been benchmarked on the same dimensions: propagation correctness, runtime overhead, and code coverage. Each evaluation confirms that the tool functions as designed, but none tests whether an attacker can defeat the tool itself. The absence of adversarial evaluation is inherited across generations of tools, producing a systematic blind spot.

Our work applies the mitigation strategy that Lecture 3 identifies for confirmation bias: explicitly exploring disconfirmatory evidence. Rather than confirming that taint propagation succeeds under benign conditions, we test whether taint metadata survives under adversarial conditions, and show that it does not for any storage strategy that keeps metadata within the process address space.

3 Guest-lecture principles

3.1 Problem-Space versus Solution-Space

Requirements engineering distinguishes between the problem-space, which captures why a system should do something and what it should achieve, and the solution-space, which specifies how it achieves it [3]. A well-structured design process begins with problem-space requirements and only then selects among solution-space alternatives. When this order is reversed, the chosen solution may fail to satisfy the actual need it was meant to address.

The DTA literature has organized metadata storage almost entirely in solution-space terms. Tools are described and compared by their implementation technique: whether they attach tags to object fields, maintain a parallel hash map, or record labels in a separate database. The problem-space question that these tools are meant to answer is whether taint metadata remains intact when the analyzed program is compromised. Our taxonomy is organized around this question rather than around implementation technique, and the answer turns on a single factor: whether metadata resides inside or outside the process address space. Viewed from this problem-space perspective, shadowing and mirroring collapse into the same category despite being different implementations, because both keep metadata reachable by any code executing within the process. Decoupling is the only strategy class where the answer changes. By shifting the organizing principle from implementation technique to security boundary, the taxonomy separates designs that can, in principle, provide metadata integrity from those that cannot.

3.2 Hidden Gaps in Implicit Assumptions

Lenberg observes that software engineering processes often rely on implicit assumptions, informal coordination, and unwritten knowledge that remain invisible as long as the operating environment stays stable [4]. When that environment changes, these hidden gaps surface as concrete failures.

DTA metadata storage designs carry an implicit assumption: that implementation complexity is a sufficient barrier to metadata tampering. Mirroring-based tools like MirrorTaint store taint labels in standalone data structures with no references from application code, and their designers treat this referential isolation as a security property in practice. This assumption has never been stated as an explicit design requirement, nor tested under adversarial conditions; it persists as unwritten knowledge inherited across tool generations.

Our cross-tool evaluation exposes this hidden gap. By constructing a heap navigation chain that traverses HotSpot's internal Metaspace structures (from Class objects through InstanceKlass to Symbol-based class name matching), we show that referential isolation does not prevent discovery. The assumption was invisible not because it was correct, but because no prior work had systematically challenged it across tools, JDK versions, and JVM configurations. Once challenged, the gap becomes visible: every in-process storage strategy we tested could be located and neutralized.

4 Data scientists, software engineers, and AI engineers

Kästner's *Machine Learning in Production* [5] describes the engineering challenges of turning ML models into deployed products. The following discusses five aspects of Chapters 1 and 2 in relation to my research.

Data scientists versus software engineers. Kästner characterizes data scientists as focused on model training and evaluation in exploratory workflows, and software engineers as focused on delivering products under constraints, with training in architecture, testing, and deployment. I agree with this distinction. The two roles reflect genuinely different priorities and educational backgrounds, and the gap between them explains many of the production failures he describes.

System-wide view. Chapter 2 argues that system-level qualities like safety emerge from interactions between ML and non-ML components, not from model accuracy alone. Kästner illustrates this with a smart toaster: a model that miscalculates toasting time could start a fire, and the fix is not a better model but system-level safety mechanisms (duration caps, temperature sensors, thermal fuses) that contain failure despite the unreliable component. This perspective applies directly to my research. DTA tools are evaluated in a model-centric way: propagation correctness, runtime overhead, and bytecode coverage confirm that the taint engine works under normal conditions. But the taint engine and the analyzed program share a process address space, so adversarial code can reach and modify taint metadata. Evaluating the engine in isolation misses this attack surface. The structure matches Kästner's toaster: the component (taint propagation) works correctly, but the system lacks a safety mechanism (process-external metadata storage) to contain failures when the component's assumptions are violated.

Role convergence. I do not think data scientists and software engineers will merge into a single role. The depth required in each area is too great. What is more realistic is Kästner's notion of T-shaped team members: deep in one discipline with working-level literacy across others. Data scientists who understand deployment write better pipeline code; software engineers who understand model behavior design better safeguards around unreliable components.

AI Engineering. Kästner notes that "AI Engineering" currently refers mostly to MLOps: building pipelines, deploying models, and monitoring them in production, which remains model-centric. If it is to become a distinct discipline, it should be understood as software engineering applied to systems

containing ML components, addressing challenges that neither traditional SE nor data science covers alone: missing specifications, data drift, pipeline automation, and the integration of learned components into larger systems.

Foundation models and shifting boundaries. Foundation models, AI coding assistants, and agentic workflows lower the barrier for software engineers to incorporate ML: writing prompts replaces training models. But system-level engineering challenges do not disappear; they change form. Instead of training data quality, engineers now face prompt injection, hallucination, context window limits, and API cost. For security research specifically, agentic workflows introduce new attack surfaces such as tool-use exploitation and trust propagation across multi-step chains. The fundamental lesson from Kästner's book holds: the hard problems live at the boundary between ML components and the rest of the system, not inside the model itself.

5 Paper analysis

5.1 RAGProbe: Breaking RAG Pipelines with Evaluation Scenarios

Core ideas. Sivasothy et al. propose evaluation scenarios as a structured method for testing Retrieval Augmented Generation (RAG) pipelines [6]. Rather than generating random question-answer pairs to check whether a RAG system responds correctly, they define a schema that specifies the document sampling strategy, chunking strategy, prompting strategy, and evaluation metrics for each scenario. They catalog six scenario types, ranging from simple numerical retrieval to multi-document compound questions to questions whose answers are absent from the corpus entirely. Evaluated against five open-source RAG pipelines, their approach triggers 77% more failures on average than the state-of-the-art baseline RAGAS. The key finding is that targeted, structurally varied evaluation scenarios expose failure modes that random testing misses. This matters for AI engineering because RAG pipelines involve many ad-hoc design decisions (chunk size, embedding model, prompt template) whose impact on system quality is difficult to assess without systematic evaluation.

Relation to my research. The structure of this work parallels my own. RAGProbe treats RAG pipelines as black boxes and systematically probes them with inputs designed to trigger failures that normal usage would not reveal. My evaluation of DTA tools follows the same logic: I treat each tool as a black box from the attacker's perspective and construct inputs (type-confusion attacks, heap navigation chains, JNI-based field writes) designed to violate security properties that normal usage would not challenge. Both works replace ad-hoc case-by-case analysis with a structured classification. RAGProbe classifies test inputs by question type to ensure coverage across failure modes; my taxonomy classifies storage strategies by security boundary to ensure coverage across defense designs. The shared principle is that structuring the evaluation space, rather than probing targets one at a time, reveals patterns that individual tests miss.

Integration into a larger project. Consider a code analysis platform that combines runtime taint tracking with a RAG-based vulnerability knowledge base. When a developer submits code, the platform runs DTA to trace how user-controlled data flows through the application, and simultaneously queries a RAG pipeline backed by CVE databases and security advisories to retrieve known vulnerabilities relevant to the detected data flows. The DTA component identifies where tainted data reaches sensitive sinks; the RAG component provides contextual information about what the vulnerability means and how to fix it. RAGProbe's evaluation scenarios would be used to validate the RAG component: ensuring it retrieves accurate vulnerability descriptions, handles compound queries (e.g., multiple CVEs in a single flow), and correctly refuses to answer when no relevant advisory exists. My research would secure the DTA component: ensuring that taint metadata cannot be tampered with by malicious code in the analyzed application.

Adaptation of my research. My current work demonstrates that specific DTA tools can be defeated, but the attacks are tool-specific. The lesson from RAGProbe is that the lasting contribution is not any single probe but the reusable schema that organizes them. To better support the evaluation-scenario approach, I could formalize my attack methodology into a reusable schema analogous to RAGProbe's evaluation scenario schema: specifying the metadata discovery strategy (heap scanning, Metaspace traversal, structural fingerprinting), the tampering strategy (direct overwrite, reference forgery, field injection), and the integrity verification method (taint propagation check before and after tampering). Such a schema would allow other researchers to systematically evaluate new DTA tools or broader classes of runtime analysis tools without redesigning attacks from scratch, shifting the contribution from individual tool-breaking results toward a general-purpose security evaluation framework.

5.2 Securing the AI Supply Chain

Core ideas. Nguyen et al. conduct a large-scale empirical study of security issues and solutions reported by developers on Hugging Face and GitHub [7]. They build a classification pipeline combining keyword matching with a fine-tuned distilBERT model to identify over 312,000 security-related discussions, then perform thematic analysis on 753 sampled posts to construct a taxonomy of 32 security issues and 24 solutions across four themes: System and Software, External Tools and Ecosystem, Model, and Data. Their key findings are that dependency incompatibilities and unsafe deserialization (particularly via Python's pickle format) are the most frequently reported issues, that security reports on community platforms far outpace formal CVE tracking, and that Model and Data themes suffer from a severe scarcity of practical solutions. Solutions are also cross-cutting: an issue in one theme is often addressed by a fix in another. This matters for AI engineering because it shows that securing AI systems requires a holistic view across the entire supply chain rather than treating each component in isolation.

Relation to my research. This paper connects to my work in two ways. First, their taxonomy captures memory corruption and arbitrary code execution as prominent security issues in AI-enabled projects, including heap-based buffer overflows and use-after-free vulnerabilities. My research targets a related class of memory-level vulnerability: type confusion in the JVM, which allows an attacker to bypass the type system, access arbitrary memory locations within the process, and tamper with taint metadata that has no application-level references. Both classes of vulnerability exploit the fact that in-process memory boundaries assumed to provide isolation can be circumvented at the memory level. Second, their finding that Model and Data security problems lack practical solutions resonates with my results. DTA tools are themselves intended as security solutions for tracking information flow, yet my work shows that the metadata these tools rely on can be silently destroyed by an in-process attacker. If a security tool can itself be defeated by the attack it is supposed to catch, it does not count as a working solution. This helps explain why Nguyen et al. find so few practical solutions for Model and Data security.

Integration into a larger project. The code analysis platform described in Section 5.1 would depend on a supply chain of third-party components: pre-trained models for the RAG-based vulnerability knowledge base, open-source DTA frameworks for runtime taint tracking, and upstream libraries that both components depend on. Nguyen et al.'s taxonomy provides a checklist for securing this supply chain. Their System and Software theme covers the platform's own code (input validation, secure deployment configuration); their External Tools and Ecosystem theme covers dependency management for the upstream libraries; and their Model and Data theme covers the integrity of the pre-trained models and vulnerability databases that the RAG component relies on. My research would contribute by ensuring that the DTA component's internal metadata is protected from tampering, addressing a security surface that falls outside Nguyen et al.'s current taxonomy because it concerns the integrity of a runtime analysis tool rather than an AI model or dataset.

Adaptation of my research. Nguyen et al. show that the AI supply chain introduces security surfaces beyond traditional code dependencies: models, datasets, and platform interactions all carry risks that developers encounter but struggle to mitigate. My current work focuses on the security of a single component (DTA metadata storage) in isolation. To better support supply chain security, I could extend my threat model to consider how DTA tools interact with the broader ecosystem they operate in. For instance, a DTA tool deployed as part of a CI/CD pipeline inherits the security posture of its upstream dependencies: a compromised library could inject code that targets the DTA tool's metadata before the tool even begins its analysis. Tracing how known CVEs propagate through dependency chains into application servers and characterizing the end impact (whether the vulnerability enables metadata tampering, taint evasion, or other integrity violations) would connect my runtime-level findings to the supply chain perspective that Nguyen et al. advocate.

6 Evidence, Originality, and GenAI-Use Transparency

Sources. Section 2 relies on course lecture slides by Feldt on verification and validation [2] and behavioral software engineering [1] (unpublished teaching materials). Section 3 relies on guest lecture slides by Frattini and Feldt on requirements engineering [3] and by Lenberg on human aspects in software engineering [4] (unpublished teaching materials). Both sections also draw on my own prior work [8] (peer-reviewed, ASE NIER 2025) and our ACSAC 2026 submission on DTA metadata security. Section 4 relies on Kästner's *Machine Learning in Production* [5], Chapters 1 and 2 (published textbook). Section 5 relies on Sivasothy et al. on RAGProbe [6] (peer-reviewed, CAIN 2025) and Nguyen et al. on AI supply chain security [7] (peer-reviewed, ICSE 2026).

Trust ranking. I trust the peer-reviewed conference papers most (ICSE, CAIN, ASE) because they have undergone formal review. The textbook is reliable for established concepts. The lecture slides I treat as introductions to known ideas rather than primary sources for technical claims. All claims about my own research are based on results I can personally defend.

GenAI use. I used Claude for brainstorming connections between lecture concepts and my research, and for language polishing and grammar checking. All analysis, claims, and final wording decisions are my own.

References

[1] R. Feldt, "Behavioral Software (and AI!?) Engineering," Lecture 3, WASP SE & Cloud Computing Course, June 2026.

[2] R. Feldt, "Quality Assurance and Testing in SE, QA for ML, ML for QA," Lecture 2, WASP SE & Cloud Computing Course, June 2026.

[3] J. Frattini, "Requirements Engineering: A Brief Introduction," Guest lecture, WASP Software Engineering and Cloud Computing, June 2026.

[4] P. Lenberg, "Importance of Human Aspects in Software Engineering," Guest lecture, WASP Software Engineering and Cloud Computing, June 2026.

[5] C. Kästner, *Machine Learning in Production: From Models to Products*, MIT Press, 2025.

[6] S. Sivasothy, S. Barnett, S. Kurniawan, Z. Rasool, and R. Vasa, "RAGProbe: Breaking RAG Pipelines with Evaluation Scenarios," in *Proc. IEEE/ACM 4th Int. Conf. AI Engineering (CAIN)*, 2025, pp. 60–71.

[7] T. A. Nguyen, T. H. M. Le, and M. A. Babar, "Securing the AI Supply Chain: What Can We Learn From Developer-Reported Security Issues and Solutions of AI Projects?," in *Proc. IEEE/ACM 48th Int. Conf. Software Engineering (ICSE)*, 2026, pp. 1–13.

[8] Y. Wu and A. Bartel, "ConfuseTaint: Exploiting Vulnerabilities to Bypass Dynamic Taint Analysis," in *Proc. IEEE/ACM 40th Int. Conf. Automated Software Engineering (ASE), NIER Track*, 2025, pp. 3907–3911.